

Apuntes_Pandas_5_Agrupación_y_agregacion_datos

May 11, 2026

1 Apuntes Pandas 5: Agrupacion y agregacion para crear filtrados de datos

Tras haber visto como podemos operar con estructuras de datos en pandas, transformar datos, indexarlos y extraerlos, el siguiente gran bloque de funcionalidades para la gestion de datos es **el filtrado y la creacion de conjuntos a partir de datos**

Hemos visto ya alguna forma de generar subconjuntos o realizar filtrados para mostrar en resultados, por ejemplo, utilizando las *maskas booleanas* o algun metodo que permita evaluar datos junto al metodo ***.drop()*** para poder un corte de los datos que queremos.

Sin embargo, aunque lo mencionamos en la primera parte de los apuntes, existen metodos especificos para realizar esto como ***groupby()*** que vamos a desggranar mejor aqui.

1.1 Definición de filtrado de datos

Se puede entende al filtrado de datos **como el conjunto de procesos que tienen como objetivo dividir nuestros conjuntos de datos en grupos disjuntos en base a algun cirterio o variable, con lo que luego poder realizar operacion o analisis**

Los “gurus” de la gestion de datos se refieren a estos procesos como “*split, apply, combine*”.

En el mundo del big data se conocen como ***map-reduce***.

1.2 Metodo basico para la agrupación y filtrado de datos: ***.groupby()***

El principal metodo que existe para agrupar datos de manera simple, en dataframes, y asi ejecutar estos procesos es ***.groupby()***

Lo que realiza es que **establece como indice la columna seleccionada y luego opera el resto de las columnas**.

Va a ser la función que nos permita **extender las posibilidades para trabajar en grupos** ya sea para calcular, filtrar, transformar,...

“dataframe”.groupby(“columna que se convertira en indice y por la que calculara”).“metodo de calculo”()

El objeto resultante de utilizar la funcion es un *DataFrameGroupBy*

```
[2]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:
↳\proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\mete
↳csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
meteo.groupby("ciudad").mean()
```

```
[2]:      año  mes    temp_c  viento_vel_kmh
ciudad
Barcelona  2015.5  6.5  17.012500      14.250000
Bilbao     2015.5  6.5  15.404167      8.862500
La Coruña  2015.5  6.5  14.412500     10.437500
Madrid     2015.5  6.5  15.637500     10.079167
Malaga     2015.5  6.5  19.083333     11.495833
Sevilla    2015.5  6.5  19.183333      9.266667
Valencia   2015.5  6.5  18.275000     10.529167
Zaragoza   2015.5  6.5  15.429167     16.670833
```

Este metodo ofrece distintas posibilidades en su redaccion que tenemos que conocer:

- Es posible seleccionar las columnas sobre las que se quiere operar indexandolas con `[]`. `> "dataframe".groupby("columna que agrupara")["columna sobre la que calculara"]."metodo de calculo"()` `> > >` Por defecto, si no se indica, la funcion realizara el calculo sobre todas las columnas

```
[3]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:
↳\proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\mete
↳csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
meteo.groupby("ciudad")["temp_c"].mean()
```

```
[3]: ciudad
Barcelona    17.012500
Bilbao       15.404167
La Coruña    14.412500
Madrid       15.637500
Malaga       19.083333
Sevilla      19.183333
Valencia     18.275000
```

```
Zaragoza      15.429167
Name: temp_c, dtype: float64
```

- Es posible seleccionar mas de 1 columna sobre las que se hara el grupo indexandolas con []. > *“dataframe”.groupby([“columna que agrupara 1”, “columna que agrupara 2”,...]).“metodo de calculo”()* > > > Esta función generara un indice jerarquico automaticamente por cada columna

```
[4]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:
↳\proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\mete
↳csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
meteo.groupby(["ciudad", "año"])["temp_c"].mean()
```

```
[4]: ciudad      año
Barcelona  2015      17.033333
          2016      16.991667
Bilbao     2015      15.608333
          2016      15.200000
La Coruña  2015      14.808333
          2016      14.016667
Madrid     2015      15.791667
          2016      15.483333
Malaga     2015      19.366667
          2016      18.800000
Sevilla    2015      19.241667
          2016      19.125000
Valencia   2015      18.225000
          2016      18.325000
Zaragoza   2015      15.366667
          2016      15.491667
Name: temp_c, dtype: float64
```

1.2.1 Metodo para calculo de grupos de datos: *aggregate()*

Este metodo nos permite calcular multiples valores agregados de forma simultanea, indicando en una lista las funciones a utilizar (ya sean preestablecidas o personalizadas)

“dataframe”.groupby(“columna que agrupara”)[[“columna sobre la que calculara 1”, “columna sobre la que calculara 2”,...]].aggregate([“metodo de calculo1”, “metodo de calculo2”,...])

Para una mejor visualizacion y agrupacion de resultados, es recomendable, utilizar indices jerarquicos por eso se hace doble []]

A la hora de definir las funciones, puedes:

- Definirlas directamente, si son personalizadas
- Mencionarlas entre comillas """ si son de la libreria
- Invocarlas mediante punto .

```
[6]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:
↳\proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\mete
↳csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
meteo.groupby("ciudad")[["temp_c", 'viento_vel_kmh' ]].aggregate(["mean", np.
↳sum, lambda x: max(x), lambda x: min(x)])
```

C:\Users\insau\AppData\Local\Temp\ipykernel_2720\4167024212.py:8: FutureWarning:
The provided callable <function sum at 0x000001ECA3FF5F80> is currently using
SeriesGroupBy.sum. In a future version of pandas, the provided callable will be
used directly. To keep current behavior pass the string "sum" instead.

```
meteo.groupby("ciudad")[["temp_c", 'viento_vel_kmh' ]].aggregate(["mean",
np.sum, lambda x: max(x), lambda x: min(x)])
```

```
[6]:
```

	temp_c				viento_vel_kmh	
	mean	sum <lambda_0>	<lambda_1>		mean	sum
ciudad						
Barcelona	17.012500	408.3	26.8	9.0	14.250000	342.0
Bilbao	15.404167	369.7	21.9	8.0	8.862500	212.7
La Coruña	14.412500	345.9	20.0	9.3	10.437500	250.5
Madrid	15.637500	375.3	29.9	4.4	10.079167	241.9
Malaga	19.083333	458.0	28.9	11.4	11.495833	275.9
Sevilla	19.183333	460.4	29.6	9.6	9.266667	222.4
Valencia	18.275000	438.6	27.9	10.1	10.529167	252.7
Zaragoza	15.429167	370.3	26.5	6.0	16.670833	400.1

	<lambda_0>	<lambda_1>
ciudad		
Barcelona	18.0	5.0
Bilbao	12.0	6.1
La Coruña	15.7	5.8
Madrid	16.6	5.4
Malaga	16.8	8.5
Sevilla	12.9	6.4
Valencia	17.3	5.0
Zaragoza	22.8	5.9

1.2.2 Metodo de filtrado de grupos de datos: *filter()*

Este metodo nos permite **decidir que datos queremos incluir en el grupo o no**, en base a **una funcion de filtrado sobre el conjunto de valores de cada grupo**.

Caracteristicas:

- Los metodos utilizados deben ser de **evaluacion o devolver un valor *booleano***.
- El resultado sera un dataframe con las mismas columnas pero **sin las filas excluidas**

Se expresa asi:

“dataframe”.groupby(“columna que agrupara”).filter(“metodo de evaluacion 1”, “metodo de evaluacion 2”, ...)

```
[31]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:
↳ \proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\mete
↳ csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
meteo.groupby("ciudad").filter(lambda x: max(x["temp_c"])<25, lambda x:
↳ min(x["temp_c"])>10).head(10)
```

```
[31]:
```

	año	mes	ciudad	temp_c	viento_vel_kmh
1	2015	1	Bilbao	9.1	8.7
2	2015	1	La Coruña	9.6	10.8
9	2015	2	Bilbao	8.0	11.5
10	2015	2	La Coruña	9.3	12.9
17	2015	3	Bilbao	10.9	8.7
18	2015	3	La Coruña	11.2	11.0
25	2015	4	Bilbao	15.8	9.9
26	2015	4	La Coruña	14.4	9.0
33	2015	5	Bilbao	16.9	10.6
34	2015	5	La Coruña	15.9	14.5

1.2.3 Metodo de transformacion de grupos de datos: *transform()*

Este metodo nos permite **mantener las filas originales y realizar calculos sobre los valores de estas**, en base a las **caracteristicas de un grupo**.

Se expresa asi:

*“dataframe”.groupby(“columna que agrupara”)[“columna sobre la que cal-
culara”].transform(“metodo de calculo”)*

```
[33]: from pathlib import Path
import numpy as np
```

```
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:\proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\meteo.csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
Z_temp_c = meteo.groupby('ciudad')['temp_c'].transform(lambda x: (x - x.mean())/x.std())
pd.concat([meteo, Z_temp_c], axis = 'columns').head()
```

```
[33]:
```

	año	mes	ciudad	temp_c	viento_vel_kmh	temp_c
0	2015	1	Barcelona	9.1	17.7	-1.402767
1	2015	1	Bilbao	9.1	8.7	-1.436302
2	2015	1	La Coruña	9.6	10.8	-1.330867
3	2015	1	Madrid	4.4	9.0	-1.405386
4	2015	1	Malaga	11.4	13.6	-1.406828

1.2.4 Metodo de creación de dataframes a traves de grupos de datos: *apply()*

Este metodo nos permite crear dataframes por cada grupo, aplicando una funcion.

Caracteristicas:

- La funcion recibira como parametro el grupo de datos
- Devolvera un nuevo dataframe con el resultado de la funcion en base al grupo

Se expresa asi:

1º Hay que definir una funcion que tomara como parametro el grupo:

```
def "metodo de calculo personalizado" (parametro):
    parametro["nombre de la columna que se creara"] = "operaciones"
    return parametro
```

En las operaciones se puede hacer referencia a las otras columnas haciendo uso de esta estructura:

```
parametro["nombre columna"]
```

2º Hay que expresarlo asi:

```
"dataframe".groupby("columna que agrupara").apply("metodo de calculo personalizado")
```

```
[37]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame
```

```
def funcion_prueba(x):
    x["media_meteo"] = x["temp_c"].mean()
    return x

Ruta_meteo = Path(r"C:\
↳ proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\mete
↳ csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
meteeo = meteo.groupby(['ciudad', "año"]).apply(funcion_prueba)
print(meteeo)
```

			año	mes	ciudad	temp_c	viento_vel_kmh	media_meteo
ciudad	año							
Barcelona	2015	0	2015	1	Barcelona	9.1	17.7	17.033333
		8	2015	2	Barcelona	9.0	15.4	17.033333
		16	2015	3	Barcelona	12.5	16.7	17.033333
		24	2015	4	Barcelona	15.0	12.8	17.033333
		32	2015	5	Barcelona	18.8	15.1	17.033333
...								
Zaragoza	2016	159	2016	8	Zaragoza	25.2	19.3	15.491667
		167	2016	9	Zaragoza	22.6	15.6	15.491667
		175	2016	10	Zaragoza	16.3	11.9	15.491667
		183	2016	11	Zaragoza	10.0	13.4	15.491667
		191	2016	12	Zaragoza	6.2	8.4	15.491667

[192 rows x 6 columns]

```
C:\Users\insau\AppData\Local\Temp\ipykernel_2720\1232962958.py:12:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns.
This behavior is deprecated, and in a future version of pandas the grouping
columns will be excluded from the operation. Either pass `include_groups=False`
to exclude the groupings or explicitly select the grouping columns after groupby
to silence this warning.
meteeo = meteo.groupby(['ciudad', "año"]).apply(funcion_prueba)
```

1.2.5 Metodos de reorganizacion de columnas y filas a traves de grupos de datos: *unstack()*, *stack()* y *pivot_table()*

Los ultimos metodos que vamos a ver son aquellos que **nos permiten reorganizar la informacion, intercambiando filas y columnas para mejorar la visualizacion y calculo**

Existen 3 metodos principales:

- ***.unstack()***: Permite tomar el nivel mas bajo o interno de un **indice jerarquico de filas, para convertirlo en columna por cada valor del indice**. Es decir, se pivota un indice de filas a uno de columnas.

“dataframe”.groupby([“columna que agrupara 1”, “columna que agrupara 2”....])[“columna sobre la que calculara”].“metodo de calculo”().unstack()

- `.stack()`: Permite tomar el nivel mas bajo o interno de un **índice jerárquico de columnas**, para convertirlo en fila por cada valor del índice. Es decir, se pivota un índice de columnas a uno de filas.

“dataframe”.groupby([“columna que agrupara 1”, “columna que agrupara 2”....])[“columna sobre la que calculara”].“metodo de calculo”().stack()

- `.pivot_table()`: Permite generar una tabla dinamica con valores agregados

“dataframe”.pivot_table(“columna sobre la que se va a realizar el calculo”, index = “nombre del indice”, columns = “nombre columna”, aggfunc = “metodo de calculo”)

```
[1]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:
↳ \proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\mete
↳ csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
meteo.groupby(["ciudad", "año"])["temp_c"].mean().unstack()
```

```
[1]: año          2015          2016
ciudad
Barcelona  17.033333  16.991667
Bilbao     15.608333  15.200000
La Coruña  14.808333  14.016667
Madrid     15.791667  15.483333
Malaga     19.366667  18.800000
Sevilla    19.241667  19.125000
Valencia   18.225000  18.325000
Zaragoza   15.366667  15.491667
```

```
[8]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:
↳ \proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\mete
↳ csv")
meteo = pd.read_csv(Ruta_meteo, sep = ";")
calc_meteo= meteo.groupby("ciudad")["temp_c", 'viento_vel_kmh' ].
↳ aggregate(["mean", np.sum, lambda x: max(x), lambda x: min(x)])
calc_meteo.stack(level=0).head(2)
```

C:\Users\insau\AppData\Local\Temp\ipykernel_6016\2619725837.py:8: FutureWarning:
The provided callable <function sum at 0x0000022920B09F80> is currently using

SeriesGroupBy.sum. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "sum" instead.

```
calc_meteo= meteo.groupby("ciudad")[["temp_c", 'viento_vel_kmh']]
    .aggregate(["mean", np.sum, lambda x: max(x), lambda x: min(x)])
```

C:\Users\insau\AppData\Local\Temp\ipykernel_6016\2619725837.py:9: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
calc_meteo.stack(level=0).head(2)
```

```
[8]:
```

		mean	sum	<lambda_0>	<lambda_1>
ciudad					
Barcelona	temp_c	17.0125	408.3	26.8	9.0
	viento_vel_kmh	14.2500	342.0	18.0	5.0

```
[10]: from pathlib import Path
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

Ruta_meteo = Path(r"C:\proyecto-faust\M02_Python_Librerias_Big_Data\UX_Material_Extra_Librerias\U10_datasets\meteo.csv")

meteo = pd.read_csv(Ruta_meteo, sep = ";")
meteo.pivot_table("temp_c", index = "año", columns = "ciudad", aggfunc = lambda x: min(x))
```

```
[10]:
```

	ciudad	Barcelona	Bilbao	La Coruña	Madrid	Malaga	Sevilla	Valencia	\
año									
2015		9.0	8.0	9.3	4.4	11.4	9.6	10.1	
2016		10.5	10.3	10.1	6.6	13.3	12.2	11.4	

	ciudad	Zaragoza
año		
2015		6.0
2016		6.2